

Cost & Build Report

What it costs to build and what it costs to run, month to month — an internal reference for the three of us building it, before we hand the finished product to AceOne Solutions. All figures anchored to real August 2026 API and infrastructure pricing.

BUILD COST

RUNNING COST (COGS)

TASK DIVISION

ADMIN COST TRACKING

HANDOVER TIMELINE

Written by: Furqan Raza

Built by: Furqan, Wahaj & Muneeb

Prepared for: AceOne Solutions

Companion to: The Complete Project Documentation & 31-Agent Specification

Date: August 22, 2026

CONTENTS

Table of Contents

01	Executive Summary	3
02	Who's Building This & How We're Splitting the Work	4
03	What It Costs to Build — 45–60 Day Plan	5
04	Running Cost — What Each Agent Costs Per Month	7
05	Infrastructure & Scaling Notes	10
06	What the Admin Panel Needs to Track	11
07	Channel Setup Costs — Twilio, Meta/WhatsApp, Email, Payments	13
08	Timeline & Handover to AceOne Solutions	14
09	Sources & Assumptions	15

یہ دستاویز صرف ہم تینوں (فرقان، واج، منیب) کے اپنے استعمال کے لیے ہے — بنانے کی لاگت، چلانے کی لاگت، اور کام کی تقسیم۔
کمپنی خود کرے گی۔ AceOne Solutions سیلرز، پرائسنگ اور مارکیٹ کے فیصلے

Executive Summary

A quick word on what this document is and isn't, because it matters: this is a **cost and build reference for the three of us** — Furqan, Wahaj, and Muneeb — the developers building this product for AceOne Solutions. It exists so we can plan our own time, understand what the product actually costs to run, and know what the admin panel needs to be able to show once we hand it over. It is **not** a sales plan, a pricing strategy, or a go-to-market pitch — none of us are working the sales side of this, and those decisions belong to AceOne Solutions once the product is in their hands. Where this report touches money, it's about *our build effort* and *the product's real running cost* — not about how anyone should sell it.

45–60 days

TARGET TO HAND OVER A
WORKING CORE

\$77/mo

RUNNING COST — SOLO-
SCALE USAGE

\$266/mo

RUNNING COST —
PRACTICE-SCALE USAGE

3

DEVELOPERS: FURQAN,
WAHAJ, MUNEEB

QUESTION	SHORT ANSWER (DETAIL IN THE SECTIONS NOTED)
What does it cost, in our time, to build this?	Mainly the three of us across 45–60 days; actual cash spend during the build is small — roughly \$300–600 in API testing credits, hosting, and tooling — Section 03.
What does the product cost to run, per client size?	~\$77/mo at Solo-scale usage, ~\$266/mo at Practice-scale, ~\$800–900/mo at the largest scale — Section 04 breaks this down per agent.
How are the three of us splitting the work?	Section 02 — each agent gets one owner, following the "one agent, one place" pattern from the companion documentation.
What does the admin panel need to be able to show, before handover?	Live cost-vs-usage per client, agent on/off toggles, and billing status — so AceOne's own team can run the business side without needing us to touch code — Section 06.
What's left to us vs. AceOne Solutions?	Ours: build it, cost it accurately, hand it over working. AceOne's: pricing, sales, marketing, and who they sell it to.

Who's Building This & How We're Splitting the Work

Three of us are on this build, all developers, all working for AceOne Solutions. The split below follows directly from the companion documentation's "one agent, one place" architecture — every agent lives in its own identically-named folder in both the backend and frontend, so assigning an agent to a person is just naming its slug. No overlap, no merge conflicts.

Furqan Raza

Architecture and documentation (this report and its companion), functionality review across all 31 agents, the admin panel's finance/cost-tracking logic, final QA, and the handover to AceOne Solutions.

Wahaj

Backend agent logic — building out the `_services.py` files per the priority order below — plus the Stripe/billing integration inside the admin panel.

Muneeb

Backend agent logic, the WhatsApp/Instagram/Twilio channel integrations, and the frontend dashboard work (doctor panel + admin panel screens).

Agent build order & ownership

This reuses the value-ranked priority order from the companion documentation (its Section 09–10) — highest-value, still-unbuilt agents first.

ORDER	AGENT	OWNER	WHY FIRST
1	Appointment Booking	Wahaj	Completes the loop the Receptionist already starts.
2	Lead Nurturing	Muneeb	Closes the social-channel gap; pairs with the WhatsApp/Instagram service files that already exist.
3	Cost Estimation	Wahaj	Medium complexity, high impact — a fast, visible win.
4	Recovery Follow-up	Muneeb	Establishes the WhatsApp check-in pattern that Medication Reminder and Wound Care Guidance can reuse.
5	Emergency Triage	Wahaj	Safety-critical — scheduled early regardless of value order, needs the most testing time of any agent.
6+	Remaining agents, by value tier	Split by category	Once the top 5 are live, split the rest — e.g. Wahaj takes Surgery Management + Consultation, Muneeb takes Post-Surgery Care + Business & Operations.

The checklist for whoever picks up an agent (e.g. `surgery_scheduling`): edit `backend/src/services/agents/surgery_scheduling_agent/surgery_scheduling_agent_services.py` — almost always the only file that changes — touch the controller only for a new operation, the router only for a new endpoint, and `frontend/src/data/agents/surgery_scheduling.ts` only for copy changes. Full detail is in the companion documentation's Section 03 and 10.

What It Costs to Build — 45–60 Day Plan

The target the three of us are working toward is **45–60 days to a working core, including the admin/finance system, ready to hand to AceOne Solutions**. This is our own internal pace target, not a client-facing deadline — nobody outside the three of us and AceOne is expecting a date.

Illustrative value of the build effort

There's no external invoice here — this is our own time. The table below just puts a market-equivalent number on that time, purely so AceOne has a reference point for the value of what's being handed over. Treat the rate as an adjustable placeholder, not a fixed fact.

LINE ITEM	BASIS	ILLUSTRATIVE VALUE
Wahaj + Muneeb — 50 build days × 7 effective hours/day each	700 dev-hours	—
At an illustrative blended rate of \$18/hr (mid-level Pakistan-market full-stack rate)	700 hrs × \$18	≈ \$12,600
Furqan — architecture, docs, admin/finance logic, QA (part-time across the window)	~150 hrs × \$25/hr	≈ \$3,750
Illustrative total build value		≈ \$16,000–17,000

Actual cash outlay during the build window is far smaller — this is the number that matters for a real budget:

LINE ITEM	MONTHLY	OVER 45–60 DAYS
Dev/staging VPS (Hetzner CX22, 2 vCPU/4GB)	\$4.35	≈ \$9
Domain + SSL (SSL free via Let's Encrypt/Caddy)	—	≈ \$12/yr
OpenAI + Mistral + Deepgram + Groq testing credits	—	≈ \$150–250
Twilio testing credit (trial numbers, test calls/SMS)	—	≈ \$50–100
Design/QA tooling (Figma free tier, browser testing)	—	\$0–30
Total realistic cash cost to build		≈ \$300–600

The honest risk to the 45–60 day timeline isn't the money, it's scope: the companion documentation shows 30 of 31 agents are still stubs. Realistically that means building out the top handful of flagship/high-value agents (Section 02's order) plus the admin/finance panel in this window — not all 31 fully polished. Better to hand AceOne *"a sellable core, end-to-end, with real cost tracking live"* on time, with the rest following after, than to slip the whole handover chasing full coverage.

Running Cost — What Each Agent Costs Per Month

This is the number that matters for building the admin panel's cost-tracking correctly: what each of the 31 agents actually costs in API calls once it's live. The method is deliberately conservative and transparent so it can be checked and adjusted, not taken on faith.

Method

Every agent is assigned to one of five cost archetypes based on what kind of AI work it does, priced at real August 2026 rates (Section 09 has every source):

ARCHETYPE	WHAT IT MODELS	COST PER INTERACTION
Voice (flagship)	A ~3-minute live phone call: Deepgram speech-to-text + Deepgram Aura text-to-speech + Groq (fast intermediate reasoning) + OpenAI GPT-5.6 Terra (final reasoning) + Twilio per-minute.	\$0.1061
Voice (short/escalation)	A ~1-minute urgent call — same stack, shorter.	\$0.0291
Vision	One photo analyzed by OpenAI's vision model (screening or healing photo).	\$0.0064
Reasoning session (OpenAI)	A genuinely complex text exchange — consultation, risk check, multi-resource scheduling — ~2,000 in / 800 out tokens on GPT-5.6 Terra.	\$0.0136
Templated text (Mistral)	Routine, high-volume text — reminders, confirmations, checklists — on Mistral Small.	\$0.0003–0.0006

Channel costs are added where relevant: a WhatsApp utility message costs ~\$0.010 (Meta's \$0.004 base + a ~\$0.006 BSP markup), a WhatsApp marketing message ~\$0.031, and an SMS ~\$0.008. Stripe's ~2.9% + \$0.30 processing fee is charged on whatever payment the clinic is collecting from its own patient, not something the product itself pays — so it's excluded from the table below.

Per-agent monthly cost at three usage scales

The product's existing pricing page already defines three scale points — Solo, Practice, and Enterprise — so the table below uses those same three as usage-volume profiles for costing purposes: **Solo** ~ a 1–2 doctor practice (~300 interactions/mo), **Practice** ~ 3–6 doctors (~3.5× Solo volume), **Enterprise** ~ a 10+ surgeon multi-location group (~10× Solo volume). These are starting assumptions for the admin panel's cost model — worth re-checking against real usage once a few agents are live.

AGENT	STACK	SOLO \$/MO	PRACTICE \$/MO	ENTERPRISE \$/MO
Receptionist	OpenAI+Deepgram+Groq+Twilio	31.84	111.42	318.36
Appointment Booking	Mistral+WhatsApp	2.56	8.97	25.64
Reschedule & Cancellation	Mistral+WhatsApp	1.03	3.59	10.26
Appointment Reminder	Mistral+SMS	3.30	11.56	33.02
Multilingual Translation	Deepgram+Groq+OpenAI	1.17	4.08	11.65
AI Consultation	OpenAI	2.04	7.14	20.40
Photo Analysis	OpenAI Vision+Cloudinary	0.51	1.79	5.12
Video Consultation	Mistral	0.01	0.05	0.13
Medical History Intake	Mistral	0.07	0.23	0.65
Risk Assessment	OpenAI	1.36	4.76	13.60

Procedure Recommendation	Mistral	0.05	0.18	0.52
Pre-Surgery Preparation	Mistral+WhatsApp	0.82	2.87	8.20
Surgery Scheduling	OpenAI	0.54	1.90	5.44
Surgeon Calendar	Mistral	0.02	0.05	0.15
Operating Room Scheduler	OpenAI	0.54	1.90	5.44
Equipment Checklist	Mistral	0.01	0.04	0.10
Implant Inventory	Mistral	0.01	0.02	0.05
Surgical Documentation	OpenAI (+Deepgram if dictated)	0.54	1.90	5.44
Recovery Follow-up	Mistral+WhatsApp	3.08	10.77	30.77
Healing Monitoring	OpenAI Vision+Cloudinary	0.96	3.36	9.60
Emergency Triage	Deepgram+Groq+OpenAI+Twilio	0.44	1.53	4.37
Medication Reminder	Mistral+SMS	2.48	8.67	24.77
Wound Care Guidance	Mistral/OpenAI	0.07	0.23	0.65
Recovery Progress Dashboard	Aggregation, no LLM	2.00	4.00	6.00
Cost Estimation	Mistral	0.09	0.30	0.87
Payment & Invoice	Stripe fee is on the clinic's collections, not COGS	0.04	0.13	0.38
Insurance Verification	OpenAI	0.82	2.86	8.16
Analytics Dashboard	OpenAI (insight gen)	0.25	0.88	2.52
Patient Feedback	Mistral+WhatsApp	2.05	7.18	20.51
Marketing Follow-up	Mistral+WhatsApp (marketing)	9.38	32.82	93.77
Lead Nurturing	OpenAI+Instagram/WhatsApp	3.36	11.76	33.60
Variable (AI + channel) subtotal		71.41	246.95	700.14
+ Infrastructure (VPS, phone numbers, storage, auth, email — see below)		5.50	18.89	140.94
TOTAL running cost / month		76.91	265.84	841.08

Infrastructure line items

ITEM	SOLO	PRACTICE	ENTERPRISE
VPS (Hetzner Cloud)	CX22 — \$4.35	CX32 — \$7.59	CX42 — \$14.49
Twilio phone numbers (\$1.15 ea/mo)	1 — \$1.15	2 — \$2.30	3 — \$3.45
Cloudinary (photo/media storage)	Free tier	Free tier	Plus plan — \$89
Clerk (staff/doctor auth)	Free tier	Free tier	~\$25 (paid tier, multi-seat)
Brevo (email)	Free (300/day)	Starter — \$9	Starter — \$9

Two costs are deliberately excluded above: (1) Meta WhatsApp Business verification has no listed fee from Meta itself, but requires legal business documents and takes several days — budget time, not money, for it (Section 07). (2) Insurance-payer API integrations (used by Insurance Verification for reconstructive/medical-necessity cases) often carry their own per-query or platform fees set by the specific clearinghouse/EHR a client uses — genuinely client-specific, to be scoped once a client's existing systems are known.

Infrastructure & Scaling Notes

Purely technical notes for whoever is deploying and scaling this — not a sales consideration, just engineering reality as usage grows.

STAGE	WHAT TO DO
Early build / Solo-scale	A 2 vCPU/4GB VPS (e.g. Hetzner CX22) is comfortably enough — the frontend is static (nginx, no Node process at runtime) and most agent services have no real dependencies yet. Uvicorn with 1–2 workers is enough for I/O-bound async work at this stage.
As more agents go live	Construct SDK clients (OpenAI, Twilio, WhatsApp, Cloudinary) <i>lazily</i> — on first use, not at app boot in a service's <code>__init__</code> , which currently runs at every app start. This keeps memory scaling with actual traffic instead of with how many of the 31 agents have real logic.
Practice-scale usage	Step up to a 4 vCPU/8GB VPS (Hetzner CX32) and start running Celery + Redis workers for anything that shouldn't block an HTTP response — reminder scheduling, bulk follow-ups, report generation.
Enterprise-scale / multi-location	An 8 vCPU/16GB VPS (Hetzner CX42) or larger. If a client has data-residency or compliance requirements, that's a genuine technical reason to isolate them onto their own VPS/database rather than shared infrastructure — a build decision, made per client's actual requirements once known.

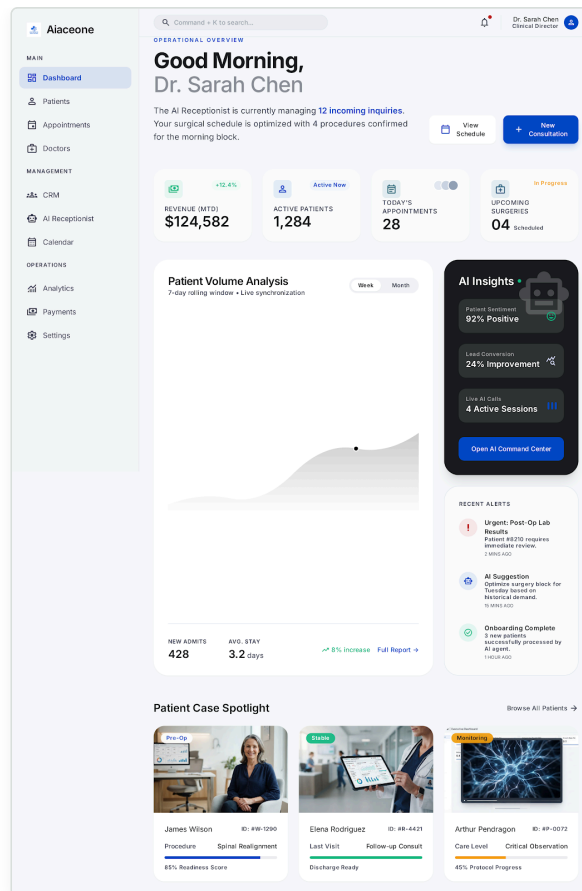
This section deliberately stays technical. Which clients actually need dedicated infrastructure, and what gets negotiated with them, is AceOne Solutions' call — our job is to make sure the architecture *supports* that choice cleanly when it's needed, which it already does via the "one agent, one place" + per-practice data model described in the companion documentation.

What the Admin Panel Needs to Track

Before handover, whoever ends up running this day-to-day at AceOne Solutions needs one place to see real cost-vs-usage without reading a line of code or asking us. That's a concrete engineering requirement for the admin panel, regardless of who staffs it later — here's what it implies we need to build:

CAPABILITY	DETAIL
Per-client agent toggles	Turn any of the 31 agents on/off for any specific client with a switch — no redeploy. The <code>AgentConfig</code> model already in the schema (companion documentation, Section 07) is the right shape for this; it just needs a real admin UI on top of it.
Billing & subscription status	Stripe-linked view of each client's plan, next billing date, and payment status.
Usage & cost visibility	For every client: how many calls/messages/photos each agent processed this month, and what that cost against Section 04's numbers — live, not a spreadsheet someone updates by hand. This is the actual "finance" piece.
Setup-fee & invoice tracking	Whatever one-time and recurring charges a client has, in one place.
Plan/feature editor	Change what's included in a client's plan from the panel — never a code change.
Message templates & thresholds	Reminder wording, Emergency Triage escalation thresholds, Recovery Follow-up cadence — editable data, not hardcoded strings in a Python file.
API key vault	OpenAI/Mistral/Deepgram/Groq/Twilio/Stripe keys managed and rotated centrally, never hardcoded per client — onboarding a new client becomes a data entry, not a deployment.
Audit log	Who (which agent, which client) did what, when — the <code>AgentLog</code> model already in the schema is the foundation, useful for debugging and for the accountability a healthcare product needs.
Access control	A client's own doctor/staff panel never shows cost or margin data — only their own usage and results. Full financial visibility is restricted to whoever AceOne designates as an admin.

Engineering implication for Wahaj & Muneeb: as each agent gets built out, anything that could plausibly change per client or per month — a price, a message template, a threshold, a feature flag — goes into a database row the admin panel can edit, not a constant in the agent's `_services.py` file. More setup work per agent up front, and it's what makes onboarding client two, three, and fifty possible without an engineer touching code each time.



Design reference — a dashboard shell like this is the right visual direction for the admin panel's home screen: whoever's running the business day-to-day needs revenue, active clients, and agent usage across every client in one view like this.

Channel Setup Costs — Twilio, Meta/WhatsApp, Email, Payments

What it actually costs to get each channel verified and running — the answer to Furqan's specific question about Twilio/Meta verification and Brevo's free email tier.

CHANNEL	SETUP PROCESS & COST	ONGOING COST
Twilio (voice + SMS)	Self-serve signup, no verification fee for a standard local number. A toll-free number needs "toll-free verification" (a short form Twilio submits to carriers) to avoid spam filtering in production — free to submit, approval in a few business days. Effectively \$0 to set up.	\$1.15/mo per local number (or \$2.00/mo toll-free) + per-minute/per-SMS usage (Section 04).
Meta WhatsApp Business API	Meta Business verification itself carries no direct fee , but requires real business documents (registration certificate, tax ID, proof of address) and typically takes a few days to a couple of weeks. Going through a BSP like Twilio adds their per-message markup (~\$0.005–0.006) but usually no separate platform fee. Effectively \$0 cash cost, budget 1–2 weeks of lead time per new client.	Per-message: ~\$0.004 utility / ~\$0.025 marketing (Meta) + ~\$0.005–0.006 BSP markup — already folded into Section 04's numbers.
Brevo (email)	Free signup. Free tier covers 300 emails/day (~9,000/month) — comfortably covers a Solo or Practice clinic's reminder/marketing email volume.	\$0 until volume exceeds the free tier, then \$9/mo (Starter) removes the daily cap.
Stripe (payments)	Free to set up, standard KYC/business verification (automated, usually same-day to a few days).	Typically ~2.9% + \$0.30 per successful card charge — the clinic's own cost of collecting patient payments, not the product's.

Bottom line: none of Twilio, Meta/WhatsApp, Brevo, or Stripe have a meaningful upfront cash cost — the real cost is *time*, specifically Meta's business-verification lead time, which should be built into onboarding for every new client rather than treated as a budget line.

Timeline & Handover to AceOne Solutions

WINDOW	FOCUS
Days 1–20	Build the flagship agents in Section 02's priority order; wire Clerk auth on the frontend.
Days 21–35	Build the admin panel's core: per-client agent toggles, Stripe billing integration, usage/cost tracking (Section 06) — the finance system this report is priced against.
Days 36–50	Doctor panel against real agent data; remaining high-value agents; one real end-to-end run-through (including an actual Twilio number and a WhatsApp verification cycle) to prove it all works together.
Days 51–60	Hardening, QA, VPS deployment, and a buffer so Meta's verification lead time (Section 07) doesn't block handover.

What "handover" actually means: a working core product (per Section 02's build order), this cost documentation, and an admin panel with real cost/usage tracking live — ready for AceOne Solutions' own team to take from there. Pricing, sales, marketing, and who they sell it to are entirely their decisions to make; our job ends at delivering something that works and is honestly costed.

This is a best-effort internal target the three of us are setting for ourselves, not a contractual deadline. If the remaining-agent scope (30 of 31 still stubs, per the companion documentation) runs long, the honest move is to say so early rather than quietly slip the date.

Sources & Assumptions

Every dollar figure in this report is built from real August 2026 published pricing, but the *volumes* (calls/month, messages/month per usage scale) are informed estimates, not measurements. Re-run the Section 04 model with real usage numbers once a client or two has a full month of data live in the admin panel — the unit prices won't change, but the volumes might, and that's what actually moves the running-cost number.

Sources

OpenAI pricing — cloudzero.com/blog/openai-pricing · Mistral pricing — mistral.ai/pricing/api · Deepgram pricing — deepgram.com/pricing · Groq pricing — cloudzero.com/blog/groq-pricing · Twilio voice/SMS pricing — automationatlas.io/twilio-pricing-explained-2026, telphiconsulting.com/twilio-phone-number-cost · WhatsApp Business API pricing — blueticks.co/whatsapp-business-api-pricing-2026, chatarmin.com/twilio-whats-app-api · Brevo pricing — automationatlas.io/brevo-pricing-explained-2026 · Hetzner Cloud VPS pricing — deployhandbook.com/pricing/hetzner · Cloudinary pricing — buildmvpfast.com/api-pricing-estimator/cloudinary · Clerk pricing — clerk.com/pricing.